MODEL

MesosGame

- players: List<Player>
- board: Board

- numPlayers: int - currentRound: int - currentEra: Era

- phase: GamePhase

- currentPlayerIndex: int -viewUpdater: ViewUpdater

// setup + setupGame(numPlayers): void
- initializePlayers(numPlayers): void
- initializeBoard(numPlayers): void
- distributeInitialFood(): void
- randomizeTurnOrder(): void

// getter (implementano GameModel) + getNumPlayers(): int + getCurrentRound(): int + getCurrentEra(): Era + getPhase(): GamePhase + getPlayers(): List<Player>

+ getBoard(): Board + getCurrentPlayer(): Player
// fase 1: piazzamento totem
+ getPlayersInTurnOrder(): List<Player>
+ getAvailableOfferTiles(): List<OfferTile>

+ placeTotem(player, offerTile): void + allTotemsPlaced(): boolean

// fase 2: risoluzione azioni + getPlayersInOfferOrder(): List<Player>
+ getAvailableCardsFromRow(row): List<Card>

+ canPlayerAffordBuilding(player, building): boolean + takeCharacterCard(player, card): void + takeBuildingCard(player, card): void + takeFood(player): void + returnTotemToTurnOrder(player): void + canPickExtraCard(player): boolean + takeExtraCard(player, card): void +notifyUpdate(GameModel): void

// fine round: eventi + getEventsToResolve(): List<EventCard>

+ resolveHunt(event): void + resolveShamanicRitual(event): void + resolveCavePainting(event): void + resolveSustenance(event): void

// fine round: gestione file e ere + discardLowerRow(): void + moveUpperRowToLower(): void + refreshUpperRow(): void + checkNewEra(): boolean- handleNewEra(): void

-handleNewEra(): void

// fine partita + isGameOver(): boolean + calculateFinalScores(): void + getWinner(): Player

Player

- playerId: int - nickname: string

- totemColor: TotemColor

- foodPoints: int - prestigePoints: int - tribe: Tribe + getFood(): int + addFood(int amount): void + removeFood(int amount): void + canAfford(int amount): bool //per capire se ha abbastanza cibo per edificio + getPrestigePoints(): int + addPrestigePoints(int amount): void + removePrestigePoints(int amount): void + getTribe(): Tribe + addCharacterCard(CharacterCard card): void + addBuildingCard(BuildingCard card): void + getTotalBuildingDiscount(): int + getTotalCharacters(): int + getTotalShamanIcons(): int

BuildingCard

- id: String

- foodCost: int - prestigeBonus: int -BuildingEffect: effect +loadFromJson(String): List<BuildingCard>

EventCard
<<abstract>>

- eventType: eventType

- isfinal: boolean + resolveEvent(List<Player> players)

OfferTrack

- tiles: List<OfferTile>

+ getTiles(): List<OfferTile> + getTileById(OfferTileId tileId): OfferTile + isTileFree(OfferTileId tileId): bool + placeTotem(Player player, OfferTileId tileId): void + getAvailableTiles(): List<OfferTile>

+ getResolutionOrder(): List<OfferTile>

+ clear(): void + containsPlayer(Player player): bool + isFull(): bool

TurnOrderTrack

//configurazione statica

- playerCount: int - foodBonusesBySlot: List<Integer>
- lastSlotFoodPenalty: int - lastSlotPrestigePenalty: int
//configurazione dinamica
- occupantsBySlot: List<Player>

+ placeInitially(List<Player> players): void + getCurrentOrder(): List<Player>

+ getFirstFreeSlotIndex(): int + returnTotem(Player player): int + getFoodBonusForSlot(int slotIndex): int + getCurrenOrder(): List<Player>

+ isLastSlot(int slotIndex): boolean + getLastSlotFoodPenalty(): int + getLastSlotPrestigePenalty(): int +placePlayer(Player player): int +getOccupantAtSlot(int): Player

GameModel <<interface>>

// setup + setupGame(numPlayers): void + getNumPlayers(): int + getCurrentRound(): int + getCurrentEra(): Era + getPhase(): GamePhase + getPlayers(): List<Player>

+ getBoard(): Board + getCurrentPlayer(): Player
// fase 1: piazzamento totem
+ getPlayersInTurnOrder(): List<Player>
+ getAvailableOfferTiles(): List<OfferTile>

+ placeTotem(player, offerTile): void + allTotemsPlaced(): boolean

// fase 2: risoluzione azioni + getPlayersInOfferOrder(): List<Player>
+ getAvailableCardsFromRow(row): List<Card>

+ canPlayerAffordBuilding(player, building): boolean + takeCharacterCard(player, card): void + takeBuildingCard(player, card): void + takeFood(player): void + returnTotemToTurnOrder(player): void + canPickExtraCard(player): boolean + takeExtraCard(player, card): void

// fine round: eventi + getEventsToResolve(): List<EventCard>

+ resolveHunt(event): void + resolveShamanicRitual(event): void + resolveCavePainting(event): void + resolveSustenance(event): void

// fine round: gestione file e ere + discardLowerRow(): void + moveUpperRowToLower(): void + refreshUpperRow(): void + checkNewEra(): boolean +notifyUpdate(GameModel): void +calculatefinalScore(): void

GamePhase <<enum>> SETUP
POSITIONING ACTION_RESOLUTION
END_ROUND GAME_OVER

Era <<enum>> ERA_1 ERA_2 ERA_3

Board

- lowerRow: List<Card>
- upperRow: List<Card>
- turnOrderTrack: TurnOrderTrack

- era2BuildingDeck: Deque<Card>
- era3BuildingDeck: Deque<Card>
- offerTrack: offerTrack
- tribeDeck: Deque<Card>

//Accesso e rimozione carte dalle file + getLowerRowCard(int index): Card + getUpperRowCard(int index): Card + removeLowerRowCard(int index): Card + removeUpperRowCard(int index): Card
//fine round + getLowerRowEvents(): List<EventCard>

+ discardLowerRowTribeCards(): void + moveUpperRowTribeCardsToLower(): void + refillUpperRow(int playerCount): List<TribeCard>

//nuova era + discardLowerRowBuildings(): void + moveUpperRowBuildingsToLower(): void + revealEraBuildings(int): void
// pesca carte tribù + fillTopRow(): void + fillBottomRow(): void
-loadBuildingDecks(int): List<BuildingCard>

+buildingCountsFor(int): int[] +loadTribeDeck(int): Card
-selectRandomBuildings(List<BuildingCard>, int): List<BuildingCard>
-dealInitialRows(int, List<BuildingCard>): void

OfferTile

- id: OfferTileId

- minPlayers: int - foodReward: int - lowerRowPicks: int - upperRowPicks: int - occupant: Player + getId(): OfferTileId + getMinPlayers(): int + getLowerRowPicks(): int + getUpperRowPicks(): int + isAvailableFor(int playerCount): bool + isFree(): bool + placeTotem(Player player): void + getOccupant(): Player + removeTotem(): Player

OfferTileId

<<enum>> A
BC
D
E

FG

TotemColor <<enum>> WHITE

BLACK
BLUE
YELLOW
RED

Tribe

- characters: List<CharacterCard>
- buildings: List<BuildingCard>

//gestione carte + addCharacter(player, card): void + addBuilding(BuildingCard): void + getCharacters(): List<CharacterCard>

+ getBuildings(): List<BuildingCard>

//conteggi per effetti + countByType(Class<CharacterCard>) : int + getMinCountAcrossAllTypes(): int + getBuilderPrestigeSum(): int + getTotalCharacters(): int + getTotalShamanIcons(): int + getTotalBuilderDiscount(): int + getCollectorDiscount(): int //raccoglitori + getDistinctInventionIcons(): int + getArtistPairs(): int + getDistinctCharacterTypesCount() : int + getPrintedBuildingPrestige(): int + getNumberOfCollectors(): int + getNumberOfHunters(): int + getShamanIcons(): int + getNumberOfArtists(): int

// trigger — iterano su buildings +-triggerOnCardAdded(player, card): void + triggerOnEvent(player, event): void + triggerOnTurnEnd(player, slotIndex): void + triggerOnGameEnd(player): int

// query aggregate su buildings + getTotalShamanIconBonus(): int + hasShamanShield(): boolean + hasShamanDouble(): boolean + getTotalSustenanceReduction(player): int +canPickExtraCard(): boolean +hasShamanGainOnTile(): boolean

TUI

BuildingEffect <<interface>>

//trigger - chiamati da Tribe + onCardAdded(Player, card): void + onEvent(player, eventCard): void + onTurnEnd(player, int): void + onGameEnd(player): int

// query — interrogate dal game loop + getShamanIconBonus(): int + preventsShamanPenalty(): boolean + doublesShamanBonus(): boolean + gainsOnShamanTie(): boolean + getSustenanceFoodReduction(player): int + canPickExtraCard(): boolean + getExtraFoodOnTurnOrder(): int
// factory + static createFrom(string, JsonNode): BuildingEffect ConfigurableEffect

//12effetti su 14

// parametri evento
- triggerEvent: EventType
- triggerTargetType: CharacterType

- foodPerUnit: int - ppPerUnit: int
// parametri sciamano
- shield: boolean
- doubleShamanBonus: boolean

- gainOnTie: boolean
- shamanExtraIcons: int

// parametri turno

- extraFoodOnTurnOrder: int - extraCardPick: boolean

// parametri sostentamento
- sustenanceType: CharacterType
- sustenanceReductionPerUnit: int

// parametri fine partita
- endGameStrategy: String

- endGameFlat: int - endGamePPPerUnit: int + onEvent(player, event): void + onGameEnd(player): int +canPickExtraCard(): boolean +preventsShamanPenalty(): boolean +doublesShamanBonus(): boolean +gainsOnShamanTie(): boolean

SetCompletionEffect

- foodReward:int -completedSets:int

//verifica set 6 tipi -> addFood + onCardAdded(player, card): void

InventorPairEffect

- foodReward: int + onCardAdded(player, characterCard): void

//verifica coppia stessa icona -> addFood

Card
<<abstract>>

- era: Era

- front: String
- back: String
- CardType: cardType

+ getFront(): String + getBack(): String + getEra(): Era

Collector

- foodDiscount: int + getFoodDiscount() Builder

- prestigePoints: int - foodBonus: int + getPrestigePoints(): int + getFoodBonus(): int

CharacterCard
<<abstract>>

- numberOfPlayers: int
// se 0 => va bene da 2 a 5

- characterType: CharacterType

-

Inventor

- icon: int + getIcon(): String Hunter

- bonus: boolean + hasBonus(): boolean Shaman Artist

- nIcons: int + getNOfIcons(): int

TurnOrderConfig

-playerCount: int -foodBonusesBySlot: List<Integer>

- lastSlotFoodPenalty: int -lastSlotPrestigePenalty: int +getPlayerCount(): int +getFoodBonusesBySlot(): List<Integer>

+getLastSlotFoodPenalty(): int +getLastSlotPrestigePenalty(): int

CharacterType <<enum>> BUILDER, HUNTER, SHAMAN,
INVENTOR, ARTIST, COLLECTOR

Sustenance EventCard

-prestigeMalus: int RitualEventCard

-prestigeMalus: int -prestigeBonus: int HuntEventCard

-prestigeBonus:int CavePainting EventCard
-bonusArtists: int -malusArtists: int -bonusPonits: int -malusPoints: int

ActionResolutionState

-currentTurnUpperTaken: int -currentTurnLowerTaken: int -currentTurnFoodTaken: boolean +onEnter(GameController): void +onExit(GameController): void +handleTakeCard(GameController, Player, TakeCardAction) +handleTakeFood(GameController, Player) +handleReturnTotem(GameController, Player) +resetTurnCounters(): void +getCurrentOfferPlayer(GameController): Player

CONTROLLER

GameState <<interface>>

-+handleAction(GameController, PlayerAction): void +onEnter(GameController): void +onExit(GameController context): void

PositioningState

-+handleAction(GameController, PlayerAction): void +onEnter(GameController context): void +onExit(GameController context): void +findOfferTile(GameModel, String): OfferTile

SetupState

-+handleAction(GameController, PlayerAction): void +onEnter(GameController): void +onExit(GameController): void

EndRoundState

-+handleAction(GameController, PlayerAction): void +onEnter(GameController): void +onExit(GameController): void

GameOverState

-+handleAction(GameController, PlayerAction): void +onEnter(GameController): void +onExit(GameController): void

States PausedState

-+handleAction(GameController, PlayerAction): void +onEnter(GameController): void +onExit(GameController): void

GameController

- model: GameModel -viewUpdater: ViewUpdate
-gameLock: ReentrantLock
-states: Map<GamePhase, GameState>
-playerMap: Map<Integer, Player>
-offerOrder: List<Player>

-offerOrderIndex: int +handleAction(PlayerAction): void +transitionTo(GamePhase): void +startGame(List<Player>)

MainController

- MIN_PLAYERS: int -MAX_PLAYERS: int -lobby: List<ClientHandler>

-clientMap: Map<Integer, ClientHandler>
- viewUpdater: ViewUpdater
- gameController: GameController
- gameInProgess: volatile boolean

-lobbyLock: Object -nextPlayerId: int -hostPlayerId: int -expectedPlayers: int +registerClient(ClientHandler): void +unregisterClient(ClientHandler): void +setExpectedPlayers(playerId, n): void +requestStartGame(): void +dispatchAction(PlayerAction): void +handleReconnection(ClientHandler, int): void +assignPlayerId(): int +isGameInProgess(): boolean +getLobbysize(): int +isHost(int): boolean +getExpectedPlayers(): int +maybeAutoStart(): void +tryStartGame(): void +broadcastLobbyUpdate(): void

PlayerState <<enum>> ACTIVE

INACTIVE

CardType <<enum>> CHARACTER, BUILDING, EVENT

EventType <<enum>>

sustenance, hunt, cavePaintings, ritual

actions PlayerAction

<<interface>>

+getPlayerID()

PlaceTotemAction

- playerId: int -offerTileId: string +getPlayerId(): int +getOfferTileId(): String

ReturnTotemAction

- playerId: int +getPlayerId(): int TakeCardAction

- playerId: int -cardIndex: int -row: CardRow

+getPlayerId(): int +getCardIndex(): int +getRow(): CardRow

TakeFoodAction

-playerId: int +getPlayerId(): int

GameEventListener <<interface>>

+onGameUpdate(NetworkMessage): void +getPlayerId(): int

Client

ViewListener <<interface>>

+updateFromView(Object)

ClientController

-serverConnection: ServerConnection
-view: View

-networkPump: Thread -running: volatile boolean +start(): void +stop(): void +pumpLoop(): void +updateFromView(Object): void

Bootstrap

- IN: bufferedReader
- DEFAULT_HOST: string

-DEFAULT_RMI_PORT: int -DEFAULT_SOCKET_PORT: int +run(): BootStrapConfig +askConnectionType(): string +askExceptedPlayers(int, int): int +askUiFlavor(): UiFlavor +askHost(): string +askPort(string): int +askNickname(): string +chooseInRange(string, int, int): int +readLine(): string

BootstrapConfig

<<record>> UiFlavor, string, int, string, string

ClientMain

+createView(UiFlavor): View

+createConnection(string[]): ServerConnection +friendlyConnectError(Throwable, string, int): string

UiFlavor <<enum>> TUI, GUI

Server ServerMain
+main(string[]): void +detectLocalIp(): string +parsePort(string): int

Connection

ClientConnection
<<interface>>

+sendMessage(Object): voi +receiveMessage(): Object +close(): void +isConnected(): boolean

ClientHandler

- connection: ClientConnection
- mainController: MainController

-playerId: int -nickname: string +run(): void +onGameUpdate(NetworkMessage): void +sendError(string): void +getPlayerId(): int +isConnected(): boolean +close(): void +attachToMatch(MainController, int): void +handlePreMatchMessage(Object): void +handleInMatchMessage(Object): void

ServerConnection
<<interface>>

+connect(string, int): void +login(string): LoginResult +setExpectedPlayers(int): void +requestStartGame(): void +sendAction(Object): void +receive(): Object +disconnect(): void +isConnected(): boolean

RMI RMIClientConnection
- remoteView: RemoteView
- inboundQueue: BlockingQueue<Object>

-active: boolean +receiveMessage(): Object +sendMessage(Object): void +close(): void +isConnected(): boolean +releaseInbound(): void +enqueueAction(PlayerAction): void

RMIClientView

-inboundQueue: BlockingQueue<NetworkMessage>

+onGameUpdate(NetworkMessage): void +take(): NetworkMessage

RemoteView
<<interface>>

+onGameUpdate(NetworkMessage): void

RMIServerConnection

-SERVICE_NAME: string
-remoteServer: RemoteServer
-clientView: RMIClientView

-playerId: int -connected: boolean +run(): void +onGameUpdate(NetworkMessage): void +sendAction(Object): void +getPlayerId(): int +isConnected(): boolean +close(): void +listMatches(): List<MatchInfo>
+createMatch(string, string): JoinResult +joinMatch(string, string): JoinResult +requestStartGame(): void +disconnect(): void RemoteServer <<interface>>
+setExpectedPlayers(int, int): void +requestStartGame(int): void +submitAction(PlayerAction): void +logout(int): void +listOpenMatches(): List<MatchInfo>

+createMatch(String, String, RemoteView): JoinResult +joinMatch(string, string, RemoteView): JoinResult RMIServer

-registry: MatchRegistry
-connection: Map<Integer, RMIClientConnection>

+login(string, RemoteView): LoginResult +setExpectedPlayers(int, int): void +requestStartGame(int): void +submitAction(PlayerAction): void +logout(int): void +createMatch(string, string,RemoteView): JoinResult +joinMatch(string, string, RemoteView): JoinResult

Socket SocketClientConnection
-socket: Socket -outStream: ObjectOutputStream
- inStream: ObjectInputStream

-connected: boolean +sendMessage(Object): void +receiveMessage(): Object +close(): void +isConnected(): boolean

SocketServerConnection

-socket: Socket -out: ObjectOutputStream
- in: ObjectInputStream

- writeLock: Object -loginQueue: BlockingQueue<LoginResult>

-connected: boolean +connect(string, int): void +login(string): LoginResult +setExpectedPlayers(int): void +requestStartGame(): void +sendAction(Object): void +receive(): Object +disconnect(): void +isConnected(): boolean +writeOnLock(Object): void +ensureConnected(): void +listMatches(): List<MatchInfo>

+createMatch(string, string): JoinResult +joinMatch(string, string): JoinResult +waitForResponse(Class<?>): Object +startReaderThread(): void SocketServer
-HANDSHAKE_TIMEOUT_MS: int -port: int -mainController:: MainController
- handshakeCounter: AtomicInteger activeHandlers: List<ClientHandler>

-serverSocket: ServerSocket -running: boolean +run(): void +handleSocket(Socket): void +stop(): void Commands LoginRequest <<record>>

- nickName: string

RequestStartGameCommand

<<record>> SetExpectedPlayersCommand

<<record>>

- n: int

VIEW

View
<<abstract>>

- listeners: List<ViewListener>
- messages: LinkedBlockingQueue<NetworkMessage>
-nickname: string

-playerId: int -gameView: GameDTO

+addListener(ViewListener): void +removeListener(ViewListener): void +notifyAllListeners(Object): void +sendAndAwaitUpdate(Object): SendResult +addMessage(NetworkMessage): void +waitForMessage():NetworkMessage +run() void +isMyturn(): boolean +maxUpperPicks(OffertileId): int +maxLowerPicks(OffetileId): int +allowsFood(OffertileId): boolean GamePrinter

-TILE_ICON: Map<OffertileId, string>

+printBoard(GameDTO, string): void +printTurnOrderTrack(List<PlayerDTO>): void +printHeader(GameDTO): void +printCardRow(string, List<CardDTO>): void +printBuildings(List<CardDTO>): void +printOffertrack(List<OfferTileDTO>, List<PlayerDTO>): void +printLeaderboard(List<PlayerDTO>, PlayerDTO, string): void +printTribe(GameDTO, string): void +printTribe(GameDTO, string, boolean): void +printRowDetails(string, List<CardDTO): void +printBuildingsDetails(GameDTO): void +iconFor(OffertileId): string +formatCardAttributes(CardDTO, boolean): string +shortLabel(CardDTO): string +formatCharacter(CardDTO, boolean): string +formatBuilding(CardDTO): string +formatEvent(CardDTO): string +filterTribe(List<CardDTO>): List<CardDTO>

+collectBuildings(List<CardDTO>, List<CardDTO>): List<CardDTO>
+resolveOccupant(int, List<PlayerDTO>): PlayerDTO
+findByNickname(GameDTO, string): PlayerDTO

+ansiFor(TotemColor): string +pad(string, int): string

TuiView

- in: Scanner
- running: boolean
-updateDispatcher: Thread
-lastErrorMessageAt: long
- pendingNotices: ConcurrentLinkedQueue<UiNotice>

-turnUpperTaken: int -turnLowerTaken: int -turnFoodTaken: boolean
-lastResolutionPlayerId: int -NoticeLevel: enum
- UiNotivr_: record

enqueueNotice(NoticeLevel, string): void
drainPendingNotices(): void
run(): void

askNickname(): void waitForLobbyStart(): void
checkQuitRequested(): boolean
runPositioningPhase(): void
ensureMyPlayerId(): boolean
runResolutionPhase(): void
findMyTile(GameDTO): OfferTileDTO
takeCardFromRow(List<CardDTO>, CardRow, string): boolean
isEvent(CardDTO): boolean
hasCharacter(List<CardDTO>): boolean
isLastReturnSlot(GameDTO): boolean
runSpectatorMenu(): void
printSpectatorOptions(): void
showOtherPlayerTribe(): void
printSpectatorBoard(GameDTO): void
startUpdateDispatcher(): void
stopUpdateDispatcher(): void
dispatchLoop(): void
applyMessage(NetworkMessage): void

announceEraOrRoundTransitions(GameDTO, GameDTO): void waitForMyTurn(): void
sendAndAwaitUpdate(Object): SendResult myPlayerId(): int
freeTiles(GameDTO): List<OfferTileDTO>
validForPlayerCount(OffertileId, int): boolean
printGameOver(GameDTO): void

chooseInRange(int, int): int readInt(int): int readLine(): string

pressEnter(): void
sleep(long): void

Printer

//all the codes for the colors printInfo(string): void

printSuccess(string): void
printError(string): void
printPrompt(string): void
printHeader(string): void
printSeparator(int): void

GUI GUIView
-LOGGER: logger connection: ServerConnection
-controller: ClientController
- primaryStage: Stage

lastErrorMessageAt: long +run(): void +showStartScreen(): void +showLobby(): void +showGame(): void +showError(string, string): void +showIErrorOnFxThread(string, string): void +showInfo(string, string): void +showInfoOnFxThread(string, string): void +waitForMessage(): NetworkMessage +sendAndAwaitUpdate(Object): SendResult +endSession(): void +shutdownAndExit(): void +showMatchSelection(): void GameScreenController

-LOGGER: Logger

-turnUpperTaken: int -turnLowerTaken: int -turnFoodTaken: boolean

-guiView: GuiView
-previousMyTurn: boolean

//all code to display the objects of the game screen +initialize(): void +initializeStaticInterface(GameDTO): void +renderFromSnapshot(GameDTO): void +updateInterface(): void +applyTotemFill(circle, totemColor): void +dispatchLoop(): void +formatResolvedEvents(EventResolvedMessage): string +addCardToRow(CardRow, CardDTO, int): void +addCardToOfferTrack(OfferTileDTO, string): void +handleOfferTrackPlacement(OfferTileDTO): void +handleCardClick(CardRow, int): void +endTurnAndReturnTotem(): void +renderTurnPill(GameDTO, boolean): void +buildPlayerCard(GameDTO, PlayerDTO, boolean, boolean): HBox

CardController

-row: int -positionIndex: int

LobbyScreenController

-connectedPlayersLabel: Label -guiView: GuiView

+initialize(): void +startDispatcher(): void +dispatchLoop(): void +renderRosetr(int,int, List<string>): void +buildPlayerRow(int, string): HBox +buildWaitingRow(): HBox

StartScreenController

- guiView: GuiView
-usernameField: TextField
- serverAddressField: TextField
- serverPortField: TextField

-connectionTypeChoice: ChoiceBox<string>

+initialize(): void +handleStartGame(): void +handleMatchSelection(ServerConnection, string): JoinResult +askNumberOfPlayers

Network

CreateMatchRequest <<record>>
-matchName: string
-nickname: string

JoinMatchRequest <<record>>
-matchId: string
-nickname: string

ListMatchesRequest <record>

ListMatchesResponse <<record>>
- matches: List<MatchInfo>

DTO

CardDTO DtoMapper GameDTO JoinResult LoginResult Matchinfo MatchStatus <enumeration> OfferTileDTO

PlayerDTO

Messages

ConnectionUpdateMessage

-nickname: string

-isConnected: boolean +getNickName(): string +isConnected(): boolean

ErrorMessage

- errorDetail: string

- error: string EventResolvedMessage
- events: List<ResolvedEvent> GameOverMessage
-endMessage: string
-finaltate: GameDTO

LobbyUpdateMessage

-currentPlayers: int -expectedPlayers: int -nicknames: List<string>

ModelUpdateMessage

-gameDTO: GameDTO

NetworkMessage <<interface>>

PhaseUpdateMessage

- newPhase: GamePhase PingMessage PongMessage RMIClientView

- currentRound: int -eraChanged: boolean
-currentEra: string TurnUpdateMessage

-activePlayerNickname: string
-activePlayerId: int

EndGameCinematic

- host : StackPane
- ranking : List<PlayerDTO>

- eraNumber : int - endMessage : String
- localPlayerId : int - onNewGame : Runnable
- onExit : Runnable

- overlay : StackPane

- centerLayer : StackPane + play() : void

- buildBackdrop() : Region
- buildEmbers() : Pane
- buildOpeningPane() : Pane
- goToLeaderboard() : void
- buildLeaderboardPane() : Pane
- buildRankingRow(int, PlayerDTO, boolean) : HBox
- statsLine(PlayerDTO) : String

- makeScroll(double, double, Node) : Scroll - unroll(Scroll, long, Runnable) : void
- cascadeIn(Node, long, double) : void
- dismiss(Runnable) : void

- parchmentLabel(String, double, FontWeight, Color) : Label - roman(int) : String

GuiAssets MatchSelectionController

- guiView: GuiView

- refreshMatches() : void
- buildMatchRow(MatchInfo) : HBox
- handleCreate() : void
- handleJoinByName() : void
- joinExisting(String) : void
- finishJoin(JoinResult, Integer) : void
- connection() : ServerConnection
- statusText(MatchStatus) : String

Persistance MatchSnapShot

- matchId : String

- expectedPlayers : int - hostNickname : String

- hostPlayerId : int - nextPlayerId : int - nickToPlayerId : Map<String, Integer>
- model : GameModel - savedAtMillis : long

PersistanceManager + fromEnvironment() : PersistenceManager + save(MatchSnapshot) : void + toBytes(MatchSnapshot) : byte[] + saveBytes(String, byte[]) : void + loadAll() : List<MatchSnapshot>

+ delete(String) : void

- tryLoad(Path) : MatchSnapshot - snapshotPath(String) : Path
- fileBase(String) : String